

数据表示与运算

有限位宽怎样保持可解释，并显式暴露信息丢失

408 · 计算机组成原理 Topic CO-01

母问题：同一串比特怎样得到确定的数学与硬件结果？

比特没有自带 signed、float 或 address 语义。完整推理链是

Bit Pattern $\rightarrow$ Interpretation $\rightarrow$ Exact Value

$\rightarrow$ Operation $\rightarrow$ Finite-width Result $\rightarrow$ Lost Information / Flags

任何答案都应先固定 (width, interpretation, operation, destination); 溢出、舍入和异常都发生在“精确数学对象重新进入有限可表示集合”的边界。

本册学习范围

- 本册解释位串如何表示整数、浮点数和校验信息，以及定长硬件如何完成加减乘除、移位和舍入。
- 题目若给出结果位宽、舍入模式或状态位定义，以题设为准；本册先把这些条件翻译成可计算的规则。
- 指令编码、完整控制通路和高级语言转换只在需要说明接口时出现；本册始终先把“位模式、数学值、目标格式”分开。

1 术语先行：读者先知道每个词在这里指什么

五个基础词

- **位串 (bit pattern)**: 由 0 和 1 组成的固定长度序列；它本身没有正负号或小数意义。
- **位宽 (width)**: 位串包含的位数。硬件只保留这个宽度内的结果，超出的位必须被截掉或另行保存。
- **无符号/有符号**: 同一位串的两种整数解释；无符号把每一位看成正的二进制权重，有符号补码把最高位看成负权。
- **ALU**: 算术逻辑单元，负责加、减、与、或、异或、比较和移位等基本运算，并可产生进位、零、负和溢出等标志。
- **IEEE 754**: 规定二进制浮点数的字段、特殊值和舍入规则的标准；本册把它当作一种格式，而不是某个处理器品牌。

目录

1 术语先行：读者先知道每个词在这里指什么	1
2 表示系统：位串、解释与可表示集合	4
3 定长加减：同一低位结果，不同溢出解释	4
4 扩展、截断与移位都在选择保留什么	4
5 乘法：完整乘积、部分积与目标宽度	5
5.1 移位加法与原码一位乘	5
5.2 Booth 的生成逻辑	5
6 除法：商余关系是最终不变量	5
7 浮点：先声明格式，再谈格点、范围与精度	6
7.1 教材自定义浮点：规格化条件来自“最高有效基数位不能冗余”	6
7.2 IEEE 754：特殊编码上的非均匀格点	6
8 浮点运算链与舍入证据	7
9 五列概念边界	7
10 做题控制协议	7
11 本册与整机的接口	8
12 知识地图：从表示到可验证结果	8
13 补充细节：从进制到可验证的位串状态机	8
13.1 进位计数制：先写位置计数式，再分离整数与小数	9
13.2 真值、机器数与四种编码：同一位串必须先声明解释	9
13.3 补码加减、ALU 与标志：低位结果和解释必须分开	10
13.4 移位：方向、补入位与取整方向共同决定语义	11
13.5 定点乘法：先保足宽，再把结果压回目标格式	11
13.6 迭代除法：恢复与不恢复共享商余不变量	11
13.7 IEEE 754：字段分配、特殊值和非均匀格点	12
13.8 浮点加减：对阶、规格化和 GRS 是连续状态	12
13.9 校验码：把合法位串做成稀疏集合	13
13.10 海明码：综合校验结果就是错误位置	13

13.11 C 类型转换：先保源值，再缩放位宽，最后按目标格式解释	13
13.12 向 CO-02 交接：大小端与对齐不是第二套数值编码	14
13.13 补充细节到微架构骨架的回接表	15
14 扩充后的自测与反向重建	15
14.1 ALU 细节：从一位全加器到可控功能单元	15
14.2 CRC 与码距的可计算边界	16

2 表示系统：位串、解释与可表示集合

一个 n 位位串只是集合 $\{0,1\}^n$ 的元素。unsigned 把它解释为

$$U(b) = \sum_{i=0}^{n-1} b_i 2^i, \quad 0 \leq U \leq 2^n - 1;$$

补码把最高位赋负权：

$$T(b) = -b_{n-1} 2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i, \quad -2^{n-1} \leq T \leq 2^{n-1} - 1.$$

同一位串可同时作为无符号数、有符号数、掩码、地址或浮点字段；意义来自消费它的操作，而不是某一位的外观。

原码把符号与绝对值分开，存在 $+0/-0$ ，加减需额外处理符号和大小。补码把定长加减统一到模 2^n 环，只有一个零，代价是范围不对称：最小负数没有同宽正数。

3 定长加减：同一低位结果，不同溢出解释

n 位加法器保留

$$r = (a + b) \bmod 2^n.$$

unsigned overflow 关注最高位 carry；signed overflow 关注精确结果是否落在补码范围。加法中“同号输入得到异号输出”可判 signed overflow；等价电路判据是符号位的进位与出位异或。

4 位运算	unsigned 解释	补码解释
1111+0001 → 0000	15 + 1 越界，carry=1	-1 + 1 = 0，无 overflow
0111+0001 → 1000	7 + 1 = 8，无 unsigned 越界	7 + 1 越界，overflow=1
1000 取负仍为 1000	位模式变换结果	-8 的同宽正数不可表示

减法先明确硬件采用 $a + (\bar{b} + 1)$ ，再按题目对 carry/borrow 的定义解释。零、负、carry、overflow 是不同谓词；不能由一个标志推出另一个解释。

4 扩展、截断与移位都在选择保留什么

zero extension 保留 unsigned 值；sign extension 保留补码值。截断到低 k 位相当于模 2^k ，通常不保留数学值。判断窄化是否无损的通用办法是：截断后按原解释扩回，是否恢复原位串/数值。

逻辑移位补零；算术右移复制补码符号位，常对应向负无穷取整的除 2^k ，不自动等于高级语言的有符号除法。左移低位结果与乘 2^k 同余，但是否溢出仍取决于目标解释和被丢高位。

舍入也不是浮点专属：只要一个更宽或更精细的定点结果要压回较窄目标格式，就必须由题设决定是截断、向最近舍入还是其他规则。浮点规格化中的两种移位要另行分清：**左规格化**通过左移消除前导冗余位，本身不会从低端丢失有效尾数；**右规格化**把有效数右移，可能把低位移出保留字段，因此这些低位必须先保留为舍入证据。规格化只是“把有效数放回合法区间”，是否发生舍入取决于这一步有没有把有效低位压出目标精度。

不变量

每次扩展、截断或移位都问：要保留的是位模式、无符号值、补码值，还是仅保留模结果？若答案不明确，后续标志判断没有语义基础。

5 乘法：完整乘积、部分积与目标宽度

两个 n 位无符号数的完整乘积最多需要 $2n$ 位；两个 n 位补码数的完整乘积也用 $2n$ 位承载。若只保留低 n 位，unsigned 与补码乘法的低位位串相同，但高位和溢出解释不同。

窄结果判据：unsigned 要求完整乘积高 n 位全零；补码要求完整乘积高 n 位等于低半部符号位的扩展。更一般地，先得到足宽精确积，再检查是否能无损压回目标格式。

5.1 移位加法与原码一位乘

无符号乘法把乘数每一位 q_i 选择的 $M \cdot 2^i$ 累加。序贯乘法器用部分积、乘数和被乘数移位，以时间换加法器/寄存器资源；并行阵列同时生成和压缩部分积，以面积与关键路径换吞吐。

原码一位乘先取符号异或，再对绝对值做无符号部分积；它不是补码 Booth 算法的同义词。题目给出联合寄存器时，必须先写每段宽度、移位方向和每拍不变量，不能跨教材套门数或节拍数。

5.2 Booth 的生成逻辑

Radix-2 Booth 用相邻位 (Q_0, Q_{-1}) 压缩连续的 1: 01 加 M , 10 减 M , 00/11 不变，然后对联合寄存器算术右移。它利用

$$00111100_2 = 01000000_2 - 00000100_2$$

把一串加法变为边界处加/减。若题目使用更高基 Booth、不同位对次序或不同寄存器布局，先声明版本；“Booth 对所有输入都更快”不是稳定结论。

6 除法：商余关系是最终不变量

除法的稳定正确性条件是

$$X = YQ + R, \quad |R| < |Y|,$$

余数符号和商的取整方向服从题设/ISA。恢复余数法在试减为负时恢复；不恢复余数法让当前余数符号决定下一拍加还是减，省去立即恢复，但结束时可能需要校正。

算法题先固定 partial remainder、quotient/dividend、divisor 和附加位布局。每拍写判断位、加减对象、移位方向和上商位；最终用商余关系回代。原码与补码算法、左移余数与右移除数版本不能混用同一口诀。

除数为零可由零检测逻辑发现，但架构结果可能是同步异常、规定值或其他语义。补码最小负数除以 -1 会产生超出同宽 signed 范围的商；“同位宽整数除法不溢出”是错误命题。

7 浮点：先声明格式，再谈格点、范围与精度

浮点数的共同抽象是

$$x = M r^e,$$

其中 r 是基数， e 决定尺度， M 决定该尺度上的有效数字。阶码位数主要购买范围，尾数位数主要购买精度，但只有在基数、阶码编码、尾数编码与规格化规则已经固定时，这句话才可直接使用；不能把 IEEE 754 的隐藏位、偏置和特殊编码自动套到教材自定义格式。

7.1 教材自定义浮点：规格化条件来自“最高有效基数位不能冗余”

遇到非 IEEE 格式，先声明

$(r, \text{exponent width/encoding, mantissa width/encoding, normalization rule})$.

对原码/符号-数值尾数，若尾数写成纯小数，规格化要求小数点后的第一个 r 进制数字非零。特别地，若 $r = 2^k$ 而机器仍用二进制写尾数，一个 r 进制数字对应 k 个二进制位，因此要检查小数点后的前 k 位是否全零；不能只看第一位。对二进制补码尾数，经典规格化判据是符号位与小数点后第一位不同，即正数形如 $0.1\dots$ ，负数形如 $1.0\dots$ 。

范围题不背某个现成端点，而从编码重新生成。先由阶码编码得到 $e_{\min}, e_{\max}$ ，再由尾数编码与规格化条件得到 M 的合法极值，最后把极值代回 $x = M r^e$ 。例如二进制补码纯小数若有 f 个小数位，则最大正尾数为 $1 - 2^{-f}$ ，最小负尾数可为 -1 ；若最大阶码为 $e_{\max}$ ，相应端点由 $M_{\min} 2^{e_{\max}}$ 与 $M_{\max} 2^{e_{\max}}$ 产生。这里的**字段角色**（阶码/尾数）与**编码方法**（补码/移码/原码）必须分层。

不能从 IEEE 反推所有浮点格式

“阶码全 0 是非规格化、全 1 是 Inf/NaN”“规格化数隐藏最高位 1”都属于 IEEE 754 的具体编码契约。题面若只给“阶码用移码、尾数用补码、基数为 2/4”，就按它给出的自定义格式求范围与规格化，不自行引入 IEEE 保留编码。

最小母例：只改基数，规格化检查位就会改变

若 $r = 4$ 、尾数采用原码纯小数，而机器仍用二进制写成 0.011011_2 ，则一个四进制数字对应两位二进制，首个基数位是 $01_2 \neq 0$ ，因此它已经规格化。若误用“二进制规格化正数第一小数位必须为 1”的口诀，就会把这个合法表示错判为非规格化。题面出现“基数 $\neq 2$ ”时，第一观察应回到一个基数位由多少机器位组成。

7.2 IEEE 754：特殊编码上的非均匀格点

普通二进制规格化数为

$$x = (-1)^s (1.f)_2 2^{E-Bias}.$$

符号选方向，阶码选尺度，尾数选该尺度上的格点。阶码全 0 或全 1 必须分支处理 zero、subnormal、infinity 和 NaN，不能套普通隐藏位公式。

subnormal 使用 $0.f$ 和最小正常指数，提供渐进下溢；infinity 与 NaN 的传播、比较和异常细节以标准/题设为准。把整个 IEEE 754 编码称作“原码”会掩盖偏置阶码、隐藏位和特殊编码。

相邻可表示数的绝对间距随指数增长，因此“大数精度更高”与“有效位数固定”不是一回事：有效相对精度近似稳定，但绝对间距变大。多数十进制小数在二进制中无限展开，输入编码时已经发生第一次舍入。

8 浮点运算链与舍入证据

加减法的生成链是

Classify → Align → Significand Operation → Normalize → Round → Encode/Signal.

对阶右移较小尾数，sticky 位应记录所有被移出低位的 OR；guard、round、sticky 与保留末位共同决定舍入。round-to-nearest, ties-to-even 下，恰好中点时让保留结果最低位为偶数；其他舍入模式必须另算。

乘除先处理符号和阶码，再算有效数，规格化、舍入并检查 exponent range。overflow 与 underflow 是目标格式边界；inexact 表示精确结果不在目标格点上。只看阶差达到尾数位数就断言小数完全无影响，可能漏掉 GRS 对最终舍入的作用。

三个浮点反例

0.1 + 0.2 涉及输入编码、运算和结果编码的多次有限映射；小量对阶后主体尾数为零，sticky 仍可能改变舍入；浮点加法不具有对所有输入成立的结合律恒等式，因为括号可能改变中间舍入点。但这不等于任意具体三元组都会得到不同结果：若两种括号下的所有中间值都能被目标格式精确表示，则本次计算仍可相等。

9 五列概念边界

概念 A	≠ 概念 B	真正区别与题面信号	混淆后果
bit pattern	≠ value	存储对象 vs 解释后的数学对象	同位串误判
carry	≠ overflow	unsigned 出位 vs signed 越界	标志互推
sign extension	≠ zero extension	保补码值 vs 保 unsigned 值	扩宽变值
full product	≠ narrow result	足宽精确积 vs 目标寄存器保留部分	漏乘法溢出
divide-by-zero detect	≠ ISA exception	电路条件 vs 软件可见契约	从门电路推出 OS 行为
rounding	≠ truncation	映射到最近/方向格点 vs 直接丢低位	浮点结果错一 ULP

10 做题控制协议

- 1. 写位宽、解释、运算和目标格式；
- 2. 先写可表示集合与精确数学结果；
- 3. 用足够中间位执行扩展、加减、部分积或商余更新；
- 4. 明确第一次丢信息的位置与舍入/截断规则；

- 5. 分开判断 carry、overflow、zero、sign、inexact 等状态；
- 6. 乘除用积回算或 $X = YQ + R$ ；浮点用分类、数量级和反向解码校验；
- 7. 最后才套 ISA 对标志、异常、饱和或返回值的契约。

压缩成第一动作

看到表示或运算题，先问：这串位按什么格式解释？精确结果在哪一步被压回多少位，丢掉了什么证据？

11 本册与整机的接口

从位串到处理器状态

本册把无语义的位串解释成给定位宽下的值、结果和信息损失证据。处理器后续会使用这些结果、宽度、符号扩展或舍入信息；某个标志是否成为软件可见异常，取决于具体指令集的规定。

本册的局部成本包括运算位宽、加法器/乘除迭代次数、关键路径、舍入逻辑和精度损失。它不直接给出程序总时间，性能题要把这些参数交给 ‘IC x CPI x clock period’ 的全局成本口径。

12 知识地图：从表示到可验证结果

考点	本册机制	接口
整数表示与运算	位权、补码、扩展、标志与目标位宽	CO-02/03
乘法器与 Booth	部分积、联合寄存器、相邻位重编码	CO-03 Use
除法与溢出	商余不变量、恢复/不恢复、边界检测	ISA exception
一般浮点 / IEEE 754	基数与编码决定规格化/范围；IEEE 再加入分类、GRS 舍入与特殊值	ISA rounding/status

一句话

有限硬件的核心不是“把数学算错”，而是按明确格式把无限或足宽结果投影回有限集合；正确答案必须同时给出保留结果、丢失信息和其软件可见解释。

本册覆盖定点与浮点表示、ALU 加法器、乘除迭代、校验码和类型转换机制；大小端与对齐只保留表示层交接，完整地址语义由 CO-02 拥有。并行阵列只是“用更多硬件换更短时间”的一种实现；Booth 或不恢复除法的寄存器安排和拍数必须以题设为准。

13 补充细节：从进制到可验证的位串状态机

本节把所有表示与运算统一到同一条可检查链上。所有计算先经过

Representation → Exact Operation → Width Reduction → Status / Error Evidence → ISA-visible Result.

其中每个箭头都对应一个可以被题目追问的状态变化：进制转换改变的是书写表示，补码运算改变的是定长环上的元素，ALU 产生的是低位结果和候选标志，浮点运算改变的是非均匀格点上的近似，校验码则改变合法码字集合的距离。

13.1 进位计数制：先写位置计数式，再分离整数与小数

任意 r 进制数的展开式为

$$x = \sum_{i=m}^n d_i r^i, \quad 0 \leq d_i < r.$$

这条式子同时处理整数位和小数位；题目中的“转换”只是选择不同的系数表示。 r 进制转十进制直接按权展开，十进制整数转 r 进制反复除以 r 取余并逆序，小数转 r 进制反复乘以 r 取整并正序。整数部分和小数部分不能混用同一方向的余数序列。

二进制与八/十六进制的分组理由

因为 $8 = 2^3$ 、 $16 = 2^4$ ，二进制小数点两侧分别从小数点向两端按 3 位或 4 位分组；左侧不足补高位零，右侧不足补低位零。补的是书写占位，不改变原数值。非 2 的幂进制没有这种机械分组，必须回到位置计数式或乘法。

有限位二进制小数是否终止，取决于约分后的分母是否只含 2 的因子。分母含 5 的十进制小数通常会在二进制中无限展开，因此即使输入阶段没有显式算术，也已经发生一次舍入；后续浮点误差不能全部归咎于最后一步加法。

13.2 真值、机器数与四种编码：同一位串必须先声明解释

对 n 位定点整数，真值是数学对象，机器数是位串。原码、反码、补码和移码分别使用不同的解释函数：

原码: $s || |x|$,

反码: $x \geq 0 \Rightarrow x, \quad x < 0 \Rightarrow$ 正数位逐位取反,

补码: $x \geq 0 \Rightarrow x, \quad x < 0 \Rightarrow 2^n + x$,

移码: 把有符号值整体加固定偏置后编码.

原码与反码存在 $+0/-0$ ；补码只保留一个零，但多出一个最小负数；移码把大小关系转成无符号字典序，常用于阶码。任何“首位是 1 所以为负数”的判断都必须先确认采用的是哪一种格式。

编码	零的数量	n 位有符号范围（常见口径）	运算/比较代价
原码	+0, -0	$-(2^{n-1} - 1) \sim +(2^{n-1} - 1)$	符号与数值分离，加减逻辑复杂
反码	+0, -0	$-(2^{n-1} - 1) \sim +(2^{n-1} - 1)$	存在循环进位，仍需处理双零
补码	一个零	$-2^{n-1} \sim +(2^{n-1} - 1)$	加减统一，溢出由目标宽度判断
移码	取决于偏置约定	通常用于指数而非一般整数算术	无符号比较可对应有符号大小

负数补码可用“按位取反加一”得到，但求相反数时必须连符号位一起处理；对最小负数求相反数仍得到自身的位串，原因是其正数不在同宽补码可表示集合内。转换题的安全流程是：先写真值，再写目标格式，最后检查是否超出目标范围；不要从一串位直接跳到十进制答案。

13.3 补码加减、ALU 与标志：低位结果和解释必须分开

在 n 位补码环上，加法器实际计算

$$S = (A + B) \bmod 2^n.$$

减法通过

$$A - B = A + \overline{B} + 1$$

复用同一加法器。低位 S 是硬件保留的结果；它是否代表正确的 unsigned 或 signed 数，要另行判断。unsigned 的 carry-out 表示最高位之外产生了进位，减法中的 borrow 取决于具体加法器约定；signed overflow 的等价判据包括“同号相加得异号”或“最高位输入进位与输出进位不同”。

ALU 可以把加法、逻辑运算、移位和比较统一成“输入位串 + 控制码 -> 结果位串 + 状态标志”的函数。常见标志的职责不同：

C ：无符号进位/借位证据， V ：有符号越界证据，
 Z ：结果全零， N/S ：结果最高位或符号证据。

题目若要求判断“是否溢出”，必须先写目标解释；不能把 $C = 1$ 当作 $V = 1$ ，也不能用结果符号位单独代替完整判据。

比较题进一步利用同一次减法的状态证据。令定长结果 $D = A - B$ ： $Z = 1$ 可以判等；对补码有符号比较，若发生 overflow，结果最高位的表面符号需要被 V 校正，因此常见 condition-code 模型用 $N \oplus V$ （或 $SF \oplus OF$ ）表示“数学意义上的 $A < B$ ”。无符号大小关系则依赖 carry/borrow 的题设定义。若题目明确采用“减法时 $CF = 1$ 表示借位”的口径，则 $A < B \Leftrightarrow CF = 1$ ， $A > B \Leftrightarrow ZF = 0 \wedge CF = 0$ ；有些 ISA 的 carry 位恰好采用“无借位”为 1，不能把这一口径跨题硬套。CO-01 只负责这些标志怎样从有限宽度结果产生，条件转移怎样消费它们由 CO-02/题设决定。

ALU 题的三层检查

第一层在位串层检查每一位和最终进位；第二层按 unsigned 解释低位结果；第三层按 signed/补码解释并判断 V 。例如两个正数相加得到最高位为 1，可能是 signed overflow，但这并不意味着 unsigned 结果非法。三层混成一步，最容易把 carry、borrow、overflow 写成同一个概念。

13.4 移位：方向、补入位与取整方向共同决定语义

逻辑左移通常左侧丢弃、右侧补零；逻辑右移左侧补零；算术右移复制符号位，以保持补码符号。对无溢出的整数，左移 k 位相当于乘 2^k ，右移 k 位只在题设允许的数值范围和取整口径下近似除以 2^k 。补码算术右移对负数趋向负无穷，而“向零截断”可能需要额外修正，不能把两者混为同一除法。

循环移位把移出的位重新接到另一端，不发生数值意义上的补零；带进位循环移位还把进位标志作为额外一位状态。判断移位是否溢出，要比较被丢弃位是否是结果符号位的合法扩展，不能只看移位后低半部。

13.5 定点乘法：先保足宽，再把结果压回目标格式

无符号乘法的精确积最多需要 $2n$ 位；补码乘法也应先在足够宽度中保留完整乘积。若目标只保留 n 位：

- unsigned 无溢出要求被丢弃的高位全为 0；
- signed 无溢出要求被丢弃的高位全为保留结果符号位的扩展。

只观察低半部或某个最终 carry，不能证明乘法没有溢出。

Booth 算法把补码乘数的连续 1 段改写为“加一次、减一次”的边界差。每一拍观察乘数当前最低位与附加位 (Q_0, Q_{-1})：01 加被乘数，10 减被乘数，00/11 不加减；随后对联合的部分积、乘数和附加位做算术右移。附加位不是装饰，它保存了上一拍的边界；算术右移则保持部分积的补码符号。最后用足宽积或十进制乘法回算，检查位序和符号。

13.6 迭代除法：恢复与不恢复共享商余不变量

除法题先固定寄存器布局、被除数/除数符号、商位方向和循环次数。最终不变量是

$$X = YQ + R,$$

并且余数满足题设的符号和范围约束。恢复余数法在每一步左移部分余数并试减除数：若结果非负，上商 1；若结果为负，则恢复原余数并上商 0。它的优点是规则直观，代价是失败时多一次恢复加法。

不恢复余数法把“本拍余数为正/负”作为下一拍动作依据：正则减除数并上 1，负则加除数并上 0；最后若余数为负，再做一次修正。它省掉了每次立即恢复，但要求严格保持符号判断、左移顺序和末位修正。两种算法的逐拍表不能混用；题目给出寄存器图时，以图中的连接和移位方向为准。

乘除迭代题的最小逐拍表

每拍至少记录五列：当前判断位/符号、加减对象、移位方向、上商或重编码位、循环后不变量。最后再做积回算或商余回算。若只能背“先移位还是先加减”的口诀而不能写出不变量，换一种寄存器布局就会失效。

13.7 IEEE 754：字段分配、特殊值和非均匀格点

普通二进制规格化数写成

$$x = (-1)^s(1.f)_22^{e-bias}.$$

阶码使用偏置表示，便于比较和处理指数范围；规格化数隐藏最高有效位 1，把有限字段留给更多有效精度。字段全零或全一时必须进入特殊分支：

<i>E</i>	<i>F</i>	含义
0	0	+0/ − 0
0	≠ 0	subnormal
中间值	任意	normal
全一	0	±∞
全一	≠ 0	NaN

408 中最常用的两种二进制格式要把“字段位数”和“有效精度”分开记：

格式	<i>S</i>	<i>E</i>	<i>F</i>	Bias	规格化有效精度
binary32 / float	1	8	23	127	1 + 23 = 24 位
binary64 / double	1	11	52	1023	1 + 52 = 53 位

对 binary32，规格化数的阶码字段只使用 $E = 1 \sim 254$ ，所以真实指数为 $-126 \sim 127$ ； $E = 0$ 与 $E = 255$ 留给零、非规格化数、无穷与 NaN 等特殊类。这里“阶码”是字段角色，“偏置表示/移码思想”是编码方法，两者不能写成同一个概念。binary64 同理，只是字段宽度和 Bias 改变。

subnormal 用固定的最小指数和 $0.f$ 提供渐进下溢；不能把它误套成隐藏位为 1 的 normal。相邻浮点格点的绝对间距随指数增大，因而相对精度大致稳定而绝对间距变粗；“指数越大精度越高”与“有效位数固定”并不矛盾。

13.8 浮点加减：对阶、规格化和 GRS 是连续状态

两个有限浮点数相加的完整顺序为

$$\text{Classify} \rightarrow \text{Align} \rightarrow \text{Significand Operation} \rightarrow \text{Normalize} \rightarrow \text{Round} \rightarrow \text{Encode}.$$

对阶时通常向较大阶码看齐，把较小尾数右移；右移丢掉的所有位先压成 sticky，再由 guard、round、sticky 和保留结果最低位决定舍入。相加可能产生最高位进位而右规，相减可能出现前导零而左规；两者都要同步修正阶码。舍入模式若是 round-to-nearest, ties-to-even，中点时保留结果最低位取偶数；题设改变舍入模式时不能继续套这个结论。

输入编码、有效数运算和结果编码都可能舍入，因此 $0.1 + 0.2$ 一类题不能只在十进制表面解释。浮点加法不满足结合律，是指不存在对全部浮点输入都成立的结合律恒等式；不同括号可能改变中间的对阶、规格化和舍入位置。对某个受限输入集合，若两种括号下所有中间精确结果都落在可表示格点上，则仍可得到相同结果。判断小量“完全无影响”也要看 sticky；即使主体尾数最终为零，被移出的低位仍可能改变最后一 ULP。

13.9 校验码：把合法位串做成稀疏集合

校验码的统一问题是：通信或存储发生错误后，接收方如何从冗余位判断“这串位是否仍是合法码字”，以及在能力范围内定位错误。码距 d_{min} 是任意两个合法码字之间的最小汉明距离；检测 t 位错误至少需要 $d_{min} \geq t + 1$ ，纠正 t 位错误至少需要 $d_{min} \geq 2t + 1$ 。

奇偶校验只给整个数据附加一个校验位，能检测奇数位翻转，不能定位错误，也不能可靠检测偶数位翻转。二维奇偶校验把行列约束叠加，可提供更强的定位能力，但具体能力仍由码的构造和错误模型决定。CRC 把位串看作二进制多项式，发送端在数据后补 r 个零并除以生成多项式，余数作为校验序列；接收端对完整码字做同一除法，余数为零才通过。CRC 的“除法”是模 2 加法，不是普通整数除法，减法等价于异或。

13.10 海明码：综合校验结果就是错误位置

若数据位数为 k 、校验位数为 r ，单错纠正要求

$$2^r \geq k + r + 1.$$

校验位放在位置 1, 2, 4, 8, ..., 其余位置放数据。第 j 个校验位负责所有位置编号的二进制表示中第 j 位为 1 的位置；编码时按偶校验填入，接收时重新计算各组校验结果并拼成 syndrome。syndrome 为 0 表示没有检测到单错，非零值直接给出出错位编号。SEC-DED 再增加总体奇偶位：可实现单错纠正和双错检测，但三位及以上错误不应超出题设能力范围。

13.11 C 类型转换：先保源值，再缩放位宽，最后按目标格式解释

转换题最容易错在把 destination 的 signedness 提前用于 source。对 408 常见的二进制补码机器模型，可把整数内部转换压成三类：

- **长 → 短**：保留目标宽度的低位，高位被截断；新最高位随后按目标类型承担新的数值/符号角色；
- **短 → 长**：先按源类型保值扩展。source unsigned 做零扩展，source signed 补码做符号扩展；
- **同宽 signed ↔ unsigned**：位模式不变，只改变解释函数。

因此 ‘signed short -> unsigned int’ 不能看到 destination 是 unsigned 就立即补 0；应先把 signed short 的值符号扩展到目标位宽，再让同一目标宽度位串按 unsigned 重新解释。这个顺序可以压成

$$\boxed{\text{Source Value} \rightarrow \text{Width Conversion} \rightarrow \text{Destination Interpretation}}.$$

整型与浮点之间还要额外比较范围和有效精度。在常见 32 位 ‘int’、binary32 ‘float’、binary64 ‘double’ 模型下：

转换	稳定判断	主要风险
‘int’ → ‘float’	float 范围足够覆盖 32 位 int，但只有 24 位有效二进制精度	可能舍入，不是所有 int 都精确
‘int’ → ‘double’	53 位有效精度足以容纳 32 位 int	可精确表示
‘float’ → ‘double’	binary64 精度与范围都更大	有限 binary32 值可精确表示
‘double’ → ‘float’	精度和范围同时缩小	可能舍入，也可能溢出
浮点 → 整数	目标格式没有小数部分	题设采用 C 常见口径且结果在目标范围内时向 0 截断；越界语义看语言/ISA/题设

显式转换的位置也不能被忽略。‘(double)(x+y)’ 先在 $x + y$ 的源类型中完成运算，再把结果转换为 double；‘(double)x+(double)y’ 则先转换两个操作数，再执行浮点加法。CO-01 只负责 “某个已经存在的源值怎样进入目标格式”；源表达式在转换前采用什么类型、溢出后语言是否仍有定义，由 CO-02 / 题设的源语言语义决定。**后面的宽类型不能反向把前面的窄类型运算变成宽运算。**

往返转换 $A \rightarrow B \rightarrow A$ 是否能恢复原值，本质上取决于第一步在当前输入集合上是否丢失信息。例如 int32 可被 binary64 精确表示，所以 ‘int32 -> double -> int32’ 在目标仍可表示时可恢复；而 ‘float -> int -> float’ 若原值含非零小数部分，第一次向整数转换已经发生截断，后续无法凭空恢复小数信息。

最稳的整数窄化检查仍是 “截后再扩”：把结果按目标解释扩回，不能恢复原值就说明转换丢失了信息。高位重复并不自动等于安全，必须相对于 source、destination 和目标位宽判断。

13.12 向 CO-02 交接：大小端与对齐不是第二套数值编码

Endian 只规定多字节对象的字节怎样映射到递增地址，不改变寄存器中的数学值，也不反转单字节内部 bit；对齐规定对象合法/高效起始地址的边界。CO-01 只输出 “对象位模式、宽度、扩展结果”，完整的地址—字节布局、结构体成员顺序、EA 与未对齐访问语义由 CO-02 拥有。

13.13 补充细节到微架构骨架的回接表

题型入口	要固定的对象	硬件需要的结果	题目验证动作
进制/编码	位串、解释函数、可表示集合	typed value、范围、比较顺序	先声明格式再读位权
补码/ALU	定长环、低位结果、 $C/V/Z/N$	结果与候选状态位	分开 unsigned/signed 判定
乘除迭代	部分积/余数、商、附加位	每拍写使能和移位需求	写循环不变量并回算
浮点	sign/exponent/fraction 特殊类	有效数、阶码、GRS、异常候选	分类后再对阶/舍入
校验码	合法码字集合、syndrome	错误检测/定位结果	用码距或多项式余数校验
C 转换	源类型、目标类型、位模式	扩展/截断控制与访存宽度	截后再扩验证
大小端/对齐	地址一字节映射与合法边界	load/store 字节选择、事务拆分	画地址表，不凭字符串顺序猜

14 扩充后的自测与反向重建

完成本册后，不能只背“公式出现在哪一节”，而应能从陌生题重新生成以下路径：

- 1. 给出一串位，先判断题设格式、位宽和目标解释；
- 2. 给出一个定点运算，保留足宽中间结果，分别判断 carry、overflow 和目标位宽截断；
- 3. 给出一个乘除迭代寄存器图，写出每拍状态与最终回算不变量；
- 4. 给出 IEEE 字段，先分类 zero/subnormal/normal/infinity/NaN，再进行运算或比较；
- 5. 给出校验位布局，按码距、生成多项式或 syndrome 判断可检测/可纠正范围；
- 6. 给出 C 转换、大小端或未对齐访问，明确位模式、地址字节和 ISA 行为是否改变；
- 7. 最后把结果封装为 CO-02/CO-03 可使用的“值、宽度、标志、异常候选”状态包。

本轮扩充的停止条件

当读者能在不查表的情况下解释“为什么这个位串这样读、哪一步丢了信息、丢失是否属于溢出/舍入/校验失败、硬件下一步需要锁存什么”，CO-01 的机制层完成；具体题型速度、公式选择和考场退出仍属于计组 Rules 与题目验证，不继续把技巧堆入本册。

14.1 ALU 细节：从一位全加器到可控功能单元

一位半加器的和与进位为

$$S = A \oplus B, \quad C = A \wedge B.$$

一位全加器再接收输入进位 C_{in} ：

$$S = A \oplus B \oplus C_{in}, \quad C_{out} = AB + AC_{in} + BC_{in}.$$

把全加器串成 n 位即可得到 ripple-carry adder；其延迟随进位逐位传播，最坏路径近似为 $O(n)$ 。先行进位把每位写成 propagate/generate：

$$P_i = A_i \oplus B_i, \quad G_i = A_i B_i, \quad C_{i+1} = G_i + P_i C_i,$$

并展开为

$$C_1 = G_0 + P_0 C_0, \quad C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0,$$

以更多并行逻辑和连线换更短的进位依赖。题目若比较串行进位与先行进位，比较的是微架构延迟、面积和扇入，不是改变 ISA 加法语义。

ALU 在加法器前可用 XOR/选择逻辑把 B 取反并把 C_0 置 1，从而复用同一硬件实现 $A - B$ ；在结果后接零检测、符号检测和溢出逻辑即可生成 Z, N, V 等 flags。逻辑运算 (AND/OR/XOR/NOT)、比较和移位可以由多路选择器接到同一结果总线；移位器是否在 ALU 内部由具体实现决定。控制字应说明功能选择、输入选择、进位输入和 flag 写使能，不能把“ALU 能算”误写成“所有 flags 每拍都更新”。

14.2 CRC 与码距的可计算边界

若数据多项式为 $M(x)$ 、生成多项式为 $G(x)$ 且 G 的次数为 r ，发送端先计算 $x^r M(x)$ 除以 $G(x)$ 的模 2 余数 $R(x)$ ，发送码字 $T(x) = x^r M(x) + R(x)$ 。接收端对 $T(x)$ 或收到的码字做同一模 2 除法，余数为零才通过。CRC 的“减法”是 XOR，不发生借位；生成多项式的首项和次数决定补零位数，不能用十进制除法替代。

若合法码字最小汉明距离为 d_{min} ，检出至多 e 位错误需 $d_{min} \geq e + 1$ ，纠正至多 t 位错误需 $d_{min} \geq 2t + 1$ ；若题目同时要求检出 e 位并纠正 t 位，常用充分条件为 $d_{min} \geq e + t + 1$ ($e \geq t$)。这些是码的结构能力，不等于某一次随机错误一定发生。

CO-01 的双重验算

算术题用十进制/数学关系回算位串，校验题用码距/多项式余数回算编码，浮点题用分类—对阶—规格化—GRS—舍入回算。若两条独立路径不一致，优先检查目标位宽、符号扩展、补零方向和编址单位，而不是直接修改最后一位答案。